Introduction

This document specifies every agent implemented in the Intellisys multi-agent research platform. Each agent is a Python class under app/agents/, sharing a common parent (BaseAgent in app/agents/base.py) that provides an LLMClient instance and a log_event() method used for the execution_logs table that powers the Live Monitor timeline.

Five agents are implemented: Supervisor, Research, Analyst, Critic, and Writer. They execute in a LangGraph-defined graph with one conditional loop (Critic → Analyst), described fully in the Architecture Documentation and Workflow Documentation.

1. Supervisor Agent

Converts a user's raw research objective into a structured, executable Research Plan consisting of 3–6 specific, non-overlapping research questions. It is the entry point of the LangGraph workflow and defines the scope of everything that follows.

Responsibilities

- Interpret the user's free-text research objective (e.g. "Should we adopt Kubernetes?").
- Decompose the objective into 3–6 distinct, non-overlapping research questions.
- Tailor question style to the nature of the topic — a technical survey is decomposed differently than a business decision.
- Attach a short rationale to each question explaining why it matters to the overall objective.
- Emit a single strict-JSON object that downstream code can parse without ambiguity.

Inputs

- objective (string) — the raw research question or task typed into the New Research form.

Outputs

- ResearchPlan (Pydantic model) — objective, a list of 3–6 {id, question, rationale} items, and optional notes.

Allowed Tools

- None. The Supervisor never calls web_search, never queries the database beyond writing execution-log entries via BaseAgent.log_event().

Allowed Actions

- Call the OpenRouter LLM via LLMClient.complete() with json_mode enabled.
- Parse and validate the LLM's response against the ResearchPlan schema.
- Log a 'started' and a 'completed' event to execution_logs.

Prohibited Actions

- Performing web searches or retrieving evidence directly.
- Writing to the evidence, reports, or tasks tables.
- Producing free-text output — the LLM is instructed to return JSON only.
- This restriction is enforced by convention (allowed_tools = ()) rather than a runtime sandbox: the agent's code simply never invokes a search function, so it cannot be tricked into acting outside its role by prompt content alone.

Prompt Strategy

The system prompt instructs the model to output only JSON matching the ResearchPlan shape, with no prose before or after.

The prompt explicitly directs the model to tailor the character of the questions to the topic type, rather than applying one fixed template — this is the deliberate fix for a previously observed 'generic report' failure mode, since a weak or generic plan propagates a weak report all the way through the pipeline.

The raw text response is passed through LLMClient.parse_json(), which strips markdown code fences before parsing, and any parse failure raises a hard error rather than falling back to default content.

Failure Conditions

- OpenRouter is not configured (missing API key) — raises LLMNotConfiguredError immediately, no retry.
- The LLM returns non-JSON or malformed JSON — parse_json() raises ReportGenerationError.
- The parsed JSON does not satisfy the ResearchPlan Pydantic schema (e.g. fewer than 3 or more than 6 questions) — validation error propagates and the run is marked failed.
- Transient network failures (timeouts, 5xx) are retried up to 3 times with exponential backoff via tenacity before being treated as a failure.

Handoff Conditions

- On successful validation, the ResearchPlan is written into the shared WorkflowState and current_stage advances to 'research'.
- Control passes unconditionally to the Research Agent — the Supervisor has no branch back to itself and no clarification loop in the current implementation (see Known Limitations).

Example Input

{ "objective": "Should we adopt Kubernetes for our backend infrastructure?" }

Example Output

{ "objective": "Should we adopt Kubernetes for our backend infrastructure?", "research_questions": [{"id": "q1", "question": "What operational complexity does Kubernetes add compared to our current deployment model?", "rationale": "Directly affects team workload and on-call burden."}, {"id": "q2", "question": "What are the cost implications of running a Kubernetes cluster at our current scale?", "rationale": "Cost is a primary decision driver for infrastructure changes."}, {"id": "q3", "question": "What migration path exists from our current architecture to Kubernetes?", "rationale": "Determines implementation risk and timeline."}, {"id": "q4", "question": "What do comparable-sized companies report as outcomes after

adopting Kubernetes?", "rationale": "Provides real-world evidence beyond vendor claims."}], "notes": "Objective framed as a binary adoption decision; questions cover cost, complexity, migration, and precedent." }

2. Research Agent

Executes web searches for every research question produced by the Supervisor and persists each result as evidence. This agent performs retrieval only — it never reasons about or interprets what it finds.

Responsibilities

- Receive the full list of research questions from the ResearchPlan.
- Issue a Tavily web search for each question, with all searches fired concurrently rather than sequentially.
- Convert each search result into an EvidenceItem (title, URL, snippet, and the question it answers).
- Persist every evidence item to the database immediately after retrieval, not batched at the end.

Inputs

- List of research questions ({id, question, rationale}) from the Supervisor's ResearchPlan.
- workflow_run_id — used to associate evidence rows with the current run.

Outputs

- A list of EvidenceItem records (title, url, snippet, question_id) written to the evidence table.
- No LLM-derived output — the Research Agent never calls OpenRouter.

Allowed Tools

- web_search (Tavily /search endpoint, wrapped in app/tools/search_tool.py).
- store_evidence() (app/tools/evidence_tool.py) — writes rows to the evidence table.

Allowed Actions

- Fire concurrent Tavily searches via asyncio.gather() — one per research question.
- Return up to 4 results per question (configurable via max_results).
- Write each EvidenceItem to the database as soon as it is retrieved.

Prohibited Actions

- Calling the LLM for any purpose — this agent performs no reasoning or summarization.
- Filtering, ranking, or interpreting search results beyond attaching them to their source question.
- Fabricating or substituting synthetic results if Tavily is unreachable — there is no fallback/mock data path; a missing TAVILY_API_KEY raises SearchNotConfiguredError immediately.

Prompt Strategy

Not applicable in the LLM sense — the Research Agent issues structured search queries (the research question text) directly to Tavily rather than constructing an LLM prompt.

Query text is passed through largely as-is from the Supervisor's question field, since the question phrasing is already optimized for retrieval by the Supervisor's own prompt design.

Failure Conditions

- TAVILY_API_KEY is missing or invalid — raises SearchNotConfiguredError with no fallback data returned.
- A network timeout or 5xx from Tavily — subject to the same tenacity retry policy as OpenRouter calls (3 attempts, exponential backoff) before surfacing as a failure.
- Zero results for a given question — evidence for that question is simply empty; this is not treated as a fatal error but is visible to the Analyst as a gap.

Handoff Conditions

- Once asyncio.gather() resolves for all questions, control advances to the Analyst node.
- Evidence already written to the database survives even if a later stage (Analyst, Critic, Writer) fails, since each evidence row is committed immediately rather than held in memory until the end of the run.

Example Input

{"question_id": "q2", "question": "What are the cost implications of running a Kubernetes cluster at our current scale?"}

Example Output

{ "question_id": "q2", "results": [{"title": "Kubernetes Cost Optimization Guide", "url": "https://example.com/k8s-cost", "snippet": "Running a managed Kubernetes cluster typically adds 15-30% overhead versus VM-based deployments at small scale..."}, {"title": "EKS Pricing Breakdown", "url": "https://example.com/eks-pricing", "snippet": "Control plane costs are fixed at $0.10/hour regardless of cluster size..."}] }

3. Analyst Agent

Reads every evidence item collected for the run and synthesizes it into structured insights, an optional comparison table, trade-offs, and explicit warnings about weak or missing evidence.

Responsibilities

- Read all Evidence rows collected so far for the current workflow run from the database.
- Number each evidence item so the LLM can cite it unambiguously in its analysis.
- Produce key insights, a comparison table (only where the topic genuinely has comparable options), trade-offs, and unsupported_claim_warnings.
- Explicitly flag evidence gaps rather than silently working around them.

Inputs

- All Evidence rows associated with the current workflow_run_id (title, url, snippet, question_id).

- The original research objective and research questions, for context.

Outputs

- AnalysisResult (Pydantic model) — key_insights, comparison_table (possibly empty), trade_offs, unsupported_claim_warnings.

Allowed Tools

- Evidence retrieval only, and strictly read-only — it queries the evidence table but never writes to it and never calls web_search itself.

Allowed Actions

- Call the OpenRouter LLM via LLMClient.complete() with the numbered evidence set as context.
- Populate comparison_table only when the topic has genuinely comparable options; otherwise return an empty list rather than a forced table.

Prohibited Actions

- Searching for additional evidence — if coverage is thin, the Analyst's job is to say so via unsupported_claim_warnings, not to compensate by fetching more.
- Fabricating a comparison table for topics without comparable options.
- Silently ignoring weak evidence — the system prompt requires honesty about claims that aren't well supported.

Prompt Strategy

Evidence is numbered in the prompt as '[1] Source: ... snippet...' so the model can refer back to specific sources by index, which the Writer later reuses for citations.

The system prompt conditions comparison_table population on the topic genuinely having comparable options, which is what prevents forced, meaningless tables on non-comparative topics (e.g. a technology survey versus a 'pick between N options' question).

The model is explicitly instructed to populate unsupported_claim_warnings for any insight not well backed by the retrieved evidence — this list is the primary input to the Critic's revision decision.

Failure Conditions

- OpenRouter is not configured or becomes unreachable mid-run — same LLMNotConfiguredError / retry-then-fail behavior as the Supervisor.
- The LLM's response fails JSON parsing or Pydantic validation against AnalysisResult.
- Evidence set is empty (all research questions returned zero results) — the Analyst can still run, but is expected to surface this as a warning rather than inventing analysis.

Handoff Conditions

- On successful validation, AnalysisResult is written to WorkflowState and control passes to the Critic node.

- On a revision loop (Critic returns REVISION_REQUIRED), control returns to this same Analyst node with the unchanged evidence set and the Critic's feedback added to context — the Research Agent is not re-invoked.

Example Input

{"evidence": [{"id": 1, "title": "Kubernetes Cost Optimization Guide", "url": "https://example.com/k8s-cost", "snippet": "...adds 15-30% overhead versus VM-based deployments..."}, {"id": 2, "title": "EKS Pricing Breakdown", "url": "https://example.com/eks-pricing", "snippet": "Control plane costs are fixed at $0.10/hour..."}]}

Example Output

{ "key_insights": ["Managed Kubernetes typically adds 15-30% infrastructure overhead versus VM-based deployments at small scale [1]."], "comparison_table": [], "trade_offs": ["Operational flexibility and portability gained; fixed control-plane cost and steeper learning curve incurred."], "unsupported_claim_warnings": ["No evidence was retrieved on team ramp-up time; this should not be treated as negligible."] }

4. Critic Agent

Reviews the Analyst's output and issues a binary verdict — APPROVED or REVISION_REQUIRED — acting as the quality gate before a report is written. Enforces a hard cap that guarantees the workflow terminates.

Responsibilities

- Serialize the AnalysisResult and evaluate it against the original objective and evidence.
- Flag concrete, specific problems (e.g. unsupported claims, missing coverage) rather than vague criticism.
- Issue a verdict of APPROVED or REVISION_REQUIRED with a justification.
- Enforce the maximum revision count so the Critic → Analyst loop cannot run indefinitely.

Inputs

- AnalysisResult JSON from the Analyst.
- revision_count — the number of times this run has already looped back from Critic to Analyst.

Outputs

- CriticFeedback (Pydantic model) — verdict (APPROVED | REVISION_REQUIRED), issues (list), justification (string).

Allowed Tools

- None. The Critic only reads the AnalysisResult JSON already in the workflow state — it cannot query the database or call web_search.

Allowed Actions

- Call the OpenRouter LLM to judge the analysis, with the system prompt framed as 'strict but fair'.
- Increment and compare revision_count against max_revisions (default 2, from Settings).

Prohibited Actions

- Overriding the hard revision cap — the cap is enforced by a plain Python conditional (if revision_count >= max_revisions: verdict = APPROVED), independent of whatever the LLM itself outputs, so the LLM cannot argue its way past it.
- Modifying the AnalysisResult directly — the Critic can only approve or request revision, never rewrite content itself.
- Accessing evidence or research tools to independently verify claims.

Prompt Strategy

System prompt frames the review as 'strict but fair,' instructing the model to flag only concrete, actionable problems rather than stylistic nitpicks.

The LLM's verdict is advisory beyond the hard cap: even if the model says REVISION_REQUIRED, the code forces APPROVED once revision_count reaches max_revisions, which is the safety mechanism preventing infinite loops described in the architecture.

Failure Conditions

- OpenRouter unreachable or misconfigured — same fail-fast/retry behavior as other LLM-calling agents.
- LLM response fails to parse into CriticFeedback schema.
- Note: reaching max_revisions is not a failure condition — it is a designed, successful termination path that forces approval.

Handoff Conditions

- route_after_critic() reads the verdict: REVISION_REQUIRED and under the cap → back to the Analyst node (evidence unchanged, fresh critique added to context).
- APPROVED, or REVISION_REQUIRED with the cap already reached → forward to the Writer node.

Example Input

{"analysis": {"key_insights": ["..."], "unsupported_claim_warnings": ["No evidence on team ramp-up time"]}, "revision_count": 0, "max_revisions": 2}

Example Output

{ "verdict": "REVISION_REQUIRED", "issues": ["Trade-offs section does not address the cost figures cited in key_insights.", "One insight has no supporting evidence citation."], "justification": "The analysis draws a conclusion in trade_offs that is not grounded in the cited evidence numbers; recommend tying the trade-off directly to evidence [1] and [2]." }

5. Writer Agent (Report Writer)

Produces the final Markdown report by combining the objective, research questions, all numbered evidence, and the Analyst's synthesis into a single adaptively-structured document, with a deterministically-built References section.

Responsibilities

- Assemble a single prompt containing the objective, research questions, all evidence (numbered [1], [2], [3]...), and the Analyst's key insights, trade-offs, comparison table, and warnings.
- Let the LLM choose the report's own structure based on what the topic needs, rather than forcing one fixed template.
- Ensure all in-text citations use the exact [n] bracket numbers already assigned to evidence.
- Deterministically build the References section from real Evidence rows — this section is never LLM-generated text.

Inputs

- Objective, ResearchPlan questions, full numbered evidence list, and the approved AnalysisResult.

Outputs

- FinalReport — Markdown report body (LLM-generated) + a References section (code-generated) — saved as a versioned row in the reports table.

Allowed Tools

- 'Export' only — turning the final text into a downloadable .md file. No search access, no further analysis capability.

Allowed Actions

- Call the OpenRouter LLM once to generate the adaptive report body.
- Programmatically append a References section built from Evidence rows: `references_lines = [f"{i+1}. [{title}]({url})" ...]`.
- Extract the report title from the LLM's own top-level '# Heading'.

Prohibited Actions

- Inventing sources in the References section — that section is not LLM output at all, so hallucinated URLs cannot appear there by construction.
- Using a fixed template of headings for every report — the system prompt explicitly requires structure to be chosen per-topic.
- Citing evidence numbers that were not actually assigned to real evidence items.

Prompt Strategy

The system prompt tells the model to choose its structure from candidates like Introduction, Background, Technical Analysis, Advantages, Limitations, Future Directions, etc., selecting only what fits the topic — this was the direct fix for a previously observed generic, one-size-fits-all report bug.

The model is told to cite using the exact [n] numbers already assigned to evidence in the prompt, and never to invent new source numbers.

The References section is deliberately kept outside the LLM's control: after the LLM returns its Markdown body, code appends a References list built directly from the Evidence rows in the database, guaranteeing every listed link was actually retrieved by Tavily.

Failure Conditions

- OpenRouter unreachable/misconfigured — same fail-fast/retry pattern as other agents.
- LLM output cannot be parsed as valid Markdown with a leading heading (title extraction fails) — surfaces as a failure rather than a silently blank title.
- Free-tier or smaller LLMs can still hallucinate citations within the body text despite numbered-evidence prompting — this is a known residual risk documented in Known Limitations, distinct from the References section which cannot be hallucinated.

Handoff Conditions

- On success, the FinalReport is saved as a new Report row (versioned) and the WorkflowRun's status is set to 'completed'.
- This is the terminal agent node in the graph — LangGraph routes writer → END with no further handoff.

Example Input

{"objective": "...", "questions": [...], "evidence": [{"n":1,"title":"...","url":"...","snippet":"..."}, ...], "analysis": {...}}

Example Output

Should We Adopt Kubernetes for Our Backend Infrastructure? ## Introduction ... ## Cost Analysis Managed Kubernetes typically adds 15-30% infrastructure overhead versus VM-based deployments at small scale [1]... ## References 1. [Kubernetes Cost Optimization Guide](https://example.com/k8s-cost) 2. [EKS Pricing Breakdown](https://example.com/eks-pricing)